



EXVUL

SMART CONTRACT **AUDIT REPORT**

Turbos Farming Smart Contract

JUNE 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Zero Balance Coin Causes Reward Extraction Deadlock	9
3.2 Hardcoded REWARD_PRECISION Causes Permanent Reward Lock for High Precision Pools	12
3.3 Incomplete Position Closure Logic Causes close_position to Always Revert	14
3.4 Non-Atomic Effective-Range Migration Preserves Stale Reward Weight	19
3.5 Emergency Withdrawal Is Disabled During Emergency State	22
3.6 Unverified sqrt_price Can Distort Farming Share Accounting	24
3.7 Emergency Withdraw Event Reports Pre-Withdrawal Vault Balance	26
3.8 Update Pool Allocate Point Event Reports Post Update Value as Old	28
3.9 Missing CollectFeeEvent Reduces Fee Collection Observability	31
3.10 Missing ClosePositionEvent Reduces Close Operation Observability	33
3.11 Missing Post-Condition Check After Adding Liquidity	35
3.12 Missing Non-Zero Amount Validation in add_liquidity_fix_coin	37
4. CONCLUSION	38
5. APPENDIX	39
5.1 Basic Coding Assessment	39
5.1.1 Apply Verification Control	39
5.1.2 Authorization Access Control	39
5.1.3 Forged Transfer Vulnerability	39
5.1.4 Transaction Rollback Attack	40
5.1.5 Transaction Block Stuffing Attack	40
5.1.6 Soft Fail Attack Assessment	40
5.1.7 Hard Fail Attack Assessment	41
5.1.8 Abnormal Memo Assessment	41
5.1.9 Abnormal Resource Consumption	41
5.1.10 Random Number Security	42

5.2 Advanced Code Scrutiny	42
5.2.1 Cryptography Security	42
5.2.2 Account Permission Control	42
5.2.3 Malicious Code Behavior	43
5.2.4 Sensitive Information Disclosure	43
5.2.5 System API	43
6. DISCLAIMER	44
7. REFERENCES	45
8. About Exvul Security	46

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Turbos Farming** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Turbos Farming	23/05/2026 - 08/06/2026	Move

Repository

<https://github.com/turbos-finance/turbos-farming>

Commit Hash

697b08719f8fdefadf0cbf85f3a37dcf821f0708

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	2
MEDIUM	4
LOW	6
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Zero Balance Coin Causes Reward Extraction Deadlock	Fixed
HIGH	Hardcoded REWARD_PRECISION Causes Permanent Reward Lock for High Precision Pools	Fixed
MEDIUM	Incomplete Position Closure Logic Causes close_position to Always Revert	Fixed
MEDIUM	Non-Atomic Effective-Range Migration Preserves Stale Reward Weight	Fixed
MEDIUM	Emergency Withdrawal Is Disabled During Emergency State	Fixed
MEDIUM	Unverified sqrt_price Can Distort Farming Share Accounting	Fixed
LOW	Emergency Withdraw Event Reports Pre-Withdrawal Vault Balance	Fixed
LOW	Update Pool Allocate Point Event Reports Post Update Value as Old	Fixed
LOW	Missing CollectFeeEvent Reduces Fee Collection Observability	Fixed
LOW	Missing ClosePositionEvent Reduces Close Operation Observability	Fixed
LOW	Missing Post-Condition Check After Adding Liquidity	Fixed
LOW	Missing Non-Zero Amount Validation in add_liquidity_fix_coin	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Zero Balance Coin Causes Reward Extraction Deadlock

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`sources/pool.move`**DESCRIPTION:**

Sui standard Programmable Transaction Block pattern for receiving tokens involves passing a zero balance coin object generated via `sui::coin::zero<T>(ctx).pool::collect_clmm_reward` accepts `mut reward_coin: Coin<RewardCoin>` parameter intended to serve as container for harvested rewards.

But when calling the underlying CLMM engine, mistakenly passed the user provided coin's current balance (`coin::value<RewardCoin>(&reward_coin)`) as `amount_max` parameter.

```
public fun collect_clmm_reward<RewardCoin, CoinA, CoinB, FeeType>(
    ...
    mut reward_coin: Coin<RewardCoin>,
    ...
): Balance<RewardCoin> {
    ...
    let collected =
        turbos_clmm::position_manager::collect_reward_with_return<
            CoinA,
            CoinB,
            FeeType,
            RewardCoin,
        >(
            clmm_pool,
            positions,
            &mut wrapped_position.nft,
            vault,
            reward_index,
            coin::value<RewardCoin>(&reward_coin), //@audit Uses
            container's existing balance as withdrawal limit
        )
    collected
}
```

```
        nft_id,  
        clk.timestamp_ms() + 60000,  
        clk,  
        versioned,  
        ctx,  
    );  
    coin::join<RewardCoin>(&mut reward_coin, collected);  
    coin::into_balance<RewardCoin>(reward_coin)  
}
```

If a user calls with a standard zero balance coin, `amount_max` evaluates to 0. underlying CLMM correctly enforces this limit and extracts exactly 0 rewards.

```
public fun collect_reward_with_return<CoinTypeA, CoinTypeB, FeeType,  
    RewardCoin>(  
    pool: &mut Pool<CoinTypeA, CoinTypeB, FeeType>,  
    positions: &mut Positions,  
    nft: &mut TurbosPositionNFT,  
    vault: &mut PoolRewardVault<RewardCoin>,  
    reward_index: u64,  
    amount_max: u64,    //@audit use as max withdrawal limit  
    recipient: address,  
    deadline: u64,  
    clock: &Clock,  
    versioned: &Versioned,  
    ctx: &mut TxContext  
): Coin<RewardCoin> {  
    ...  
  
    assert!(reward_index <  
vector::length(&position.reward_infos), EInvalidRewardIndex);  
    let reward_info = vector::borrow_mut(&mut position.reward_infos,  
reward_index);  
    let amount_collect = if (amount_max > reward_info.amount_owed)  
reward_info.amount_owed else amount_max; //@audit use as max  
withdrawal limit
```

IMPACT:

Any user who does not already hold a balance of the reward token will be permanently unable to harvest their rewards. The system will silently process the transaction and return 0 tokens. To bypass this,

users are forced to acquire the reward token from a secondary market simply to provide an arbitrary extraction limit.

RECOMMENDATIONS:

Pass the maximum possible `u64` value to allow full extraction of pending rewards.

```
public fun collect_clmm_reward<RewardCoin, CoinA, CoinB, FeeType>(
    ...
): Balance<RewardCoin> {
    ...
    let collected =
    turbos_clmm::position_manager::collect_reward_with_return_<...>(
        clmm_pool,
        positions,
        &mut wrapped_position.nft,
        vault,
        reward_index,
-       coin::value<RewardCoin>(&reward_coin),
+       0xFFFFFFFFFFFFFFFF,
        nft_id,
        clk.timestamp_ms() + 60000,
        clk,
        versioned,
        ctx,
    );
    ...
}
```

3.2 Hardcoded REWARD_PRECISION Causes Permanent Reward Lock for High Precision Pools

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`sources/pool.move, sources/rewarder.move`**DESCRIPTION:**

`turbos-farming` employs a value based share calculation. `calculate_position_share` calculates the share of a position in a pool.

```
public fun calculate_position_share(...): u128 {
    ...
    //@audit 1. Retrieve the amount of Coin A and Coin B contained in
    the user's liquidity
    let (am_a, am_b) =
    math_liquidity::get_amount_for_liquidity(sqrt_price, sa, sb,
    liquidity);

    //@audit 2. Convert Coin A's amount to Coin B equivalent using
    price + inherent Coin B amount
    (((am_a as u256) * full_math_u128::full_mul(sqrt_price,
    sqrt_price) >> 128) as u128) + (am_b as u128)
}
```

The unit of `pool_share` here is equivalent to Coin B's base quantity Base Units. If Coin B is a high precision token, the denominator `pool.total_share` will reach astronomical levels. For example, if a pool has \$10,000 USD worth of liquidity consisting of PEPE tokens (10 billion PEPE), and PEPE has 18 decimals, the total share easily reaches `1e28` base units.

When updating `acc_per_share` in `rewarder.move`'s `accumulate_pool_reward` function, hardcoded the constant `const REWARD_PRECISION: u128 = 10000000000000; (1e12)`. Even if the reward emission is configured to 1 SU1/second (`1e21` Base Units), the division calculation will trigger severe truncation:

```
fun accumulate_pool_reward(...): u128 {
    ...
}
```

```
    let reward_delta =
        full_math_u128::full_mul(rewarder.emission_per_second, ((now
- last_reward_time) as u128)) * (pool_info.allocate_point as u256) /
(rewarder.total_allocate_point as u256);

    //@audit 3. This successfully adds the reward (e.g., 1e21) to the
pool's total liabilities
    pool_info.reward_released = pool_info.reward_released +
(reward_delta as u128);

    //@audit 4. Fatal division: 1e21 / 1e28 = 0
    pool_info.acc_per_share = pool_info.acc_per_share + (reward_delta
as u128) / pool_share;
    ...
}
```

IMPACT:

Regardless of how much liquidity users provide to the high precision pool, when they call `harvest`, because `acc_per_share` is permanently stuck at 0, the system calculates their earned rewards (`ri.reward = info.share * acc_per_share`) as 0 indefinitely.

Admin will watch the genuine tokens they deposited into the Reward Vault diminish, and the `reward_released` ledger accurately incrementing, yet no user can actually withdraw the funds.

RECOMMENDATIONS:

Introduce a higher precision multiplier to defend against large denominators.

```
fun accumulate_pool_reward<RewardCoin>(...) {
    ...
    pool_info.reward_released = pool_info.reward_released +
(reward_delta as u128);
-     pool_info.acc_per_share = pool_info.acc_per_share + (reward_delta
as u128) / pool_share;
+     let scaled_delta = (reward_delta as u256) * ACC_PRECISION;
+     pool_info.acc_per_share = pool_info.acc_per_share +
((scaled_delta / (pool_share as u256)) as u128);
    ...
}
```

3.3 Incomplete Position Closure Logic Causes close_position to Always Revert

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

sources/pool.move

DESCRIPTION:

When decreasing liquidity, the underlying CLMM engine accumulates pending trading fees and CLMM rewards directly into the position's state. `pool::close_position` fails to collect these fees and rewards before calling `burn`.

```
public fun close_position<CoinA, CoinB, FeeType>(...): (Balance<CoinA>,
  Balance<CoinB>) {
  ...
  let mut nft = withdraw(global_config, rewarder_manager, pool,
    wrapped_position, clk);
  ...
  let (ba, bb) = if (pos_liquidity > 0) {
    let (ca, cb) =
      turbos_clmm::position_manager::decrease_liquidity_with_return_<...>(...);
    //@audit Accrues fees and rewards into position state
    (coin::into_balance<CoinA>(ca), coin::into_balance<CoinB>(cb))
  } else { (balance::zero<CoinA>(), balance::zero<CoinB>()) };

  turbos_clmm::position_manager::burn<CoinA, CoinB, FeeType>(
    //@audit Reverts because position state is not cleared
    positions,
    nft,
    versioned,
    ctx,
  );
  (ba, bb)
}
```

```
public fun decrease_liquidity_with_return_<CoinTypeA, CoinTypeB, FeeType>(
  ...
```

```
    ): (Coin<CoinTypeA>, Coin<CoinTypeB>) {
        ...
        let (amount_a, amount_b) = pool::burn(
            pool,
            position_owner,
            position.tick_lower_index,
            position.tick_upper_index,
            liquidity,
            clock,
            ctx,
        );
        ...
    }

    public(friend) fun burn<CoinTypeA, CoinTypeB, FeeType>(
        ...
    ): (u64, u64) {
        ...
        let (amount_a, amount_b) = modify_position(
            pool,
            owner,
            tick_lower_index,
            tick_upper_index,
            i128::neg_from(liquidity_delta),
            clock,
            ctx
        );
        ...
    }

    fun modify_position<CoinTypeA, CoinTypeB, FeeType>(
        ...
    ): (I128, I128){
        ...
        update_position(
            pool,
            owner,
            tick_lower_index,
            tick_upper_index,
            liquidity_delta,
            clock,
            ctx,
        );
    }
```

```
    ...
}

fun update_position<CoinTypeA, CoinTypeB, FeeType>(
    ...
) {
    ...
    update_position_metadata(
        pool,
        key,
        liquidity_delta,
        fee_growth_inside_a,
        fee_growth_inside_b,
        reward_growths_inside,
        ctx
    );
    ...
}

fun update_position_metadata<CoinTypeA, CoinTypeB, FeeType>(
    ...
) {
    ...
    if (tokens_owed_a > 0 || tokens_owed_b > 0) {
        // overflow is acceptable, have to withdraw before you hit
        type(uint128).max fees
        position.tokens_owed_a =
        math_u64::wrapping_add(position.tokens_owed_a, tokens_owed_a);
        position.tokens_owed_b =
        math_u64::wrapping_add(position.tokens_owed_b, tokens_owed_b);
    };
}
```

underlying CLMM [burn](#) strictly asserts that the position is completely cleared of any owed tokens.

```
public entry fun burn<CoinTypeA, CoinTypeB, FeeType>(
    ...
) {
    ...
    let position = dof::borrow_mut<address, Position>(&mut
    positions.id, nft_address);
    ...
    assert!(position.liquidity == 0 && position.tokens_owed_a == 0 &&
```



```
position.tokens_owed_b == 0, EPositionNotCleared); //@audit Aborts
because tokens_owed > 0

    let i = 0;
    let len = vector::length(&position.reward_infos);
    while (i < len) {
        let reward_info = vector::borrow(&position.reward_infos, i);
        assert!(reward_info.amount_owed == 0, EPositionNotCleared);
    //@audit Aborts if CLMM rewards are owed
        i = i + 1;
    };
    ...
}
```

IMPACT:

Any user attempting to close an active farming position that has earned any trading fees or CLMM rewards will encounter a transaction revert `EPositionNotCleared`. `close_position` is completely broken.

RECOMMENDATIONS:

Collect all pending fees and CLMM rewards after decreasing liquidity, and combine them with the principal before burning the NFT.

Follow-up fix review note:

<https://github.com/turbos-finance/turbos-farming/commit/697b08719f8fdefadf0cbf85f3a37dcf821f0708>

`collect_with_return_` withdraws unclaimed assets, the fix is incomplete: `close_position` still reverts on pools that have rewards.

```
let (fee_a, fee_b) =
    turbos_clmm::position_manager::collect_with_return_<...>(...);
...
turbos_clmm::position_manager::burn<CoinA, CoinB, FeeType>(...);
```

Underlying position state splits into trading fees `tokens_owed` and token rewards `amount_owed`.

1. `collect_with_return_` only collects `tokens_owed` and does not handle `amount_owed`.

2. `decrease_liquidity_with_return_` implicitly triggers `modify_position`, accrues new token rewards into `amount_owed`.
3. `burn` asserts `assert!(reward_info.amount_owed == 0, EPositionNotCleared)`.

As long as the pool has token rewards, `amount_owed` will be greater than zero at close time, so `burn` fails the assertion.

```
public entry fun burn<CoinTypeA, CoinTypeB, FeeType>(
    ...
) {
    ...
    let position = dof::borrow_mut<address, Position>(&mut
positions.id, nft_address);
    ...
    assert!(position.liquidity == 0 && position.tokens_owed_a == 0 &&
position.tokens_owed_b == 0, EPositionNotCleared); //@audit-ok Fixed

    let i = 0;
    let len = vector::length(&position.reward_infos);
    while (i < len) {
        let reward_info = vector::borrow(&position.reward_infos, i);
        assert!(reward_info.amount_owed == 0, EPositionNotCleared);
//@audit Aborts if CLMM rewards are owed, reverted
        i = i + 1;
    };
    ...
}
```

Additional workaround guidance:

Generic constraints prevent `close_position<CoinA, CoinB, FeeType>` from calling the underlying `collect_reward<..., RewardCoin>` without an explicit `RewardCoin` type parameter. The contract cannot dynamically clear all reward types.

<https://docs.sui.io/develop/transactions/ptbs/prog-txn-blocks>

Consider using a PTB to work around reward accrual. The frontend can build a PTB:

1. `router::collect_clmm_reward<T>` (call as many times as needed to drain every reward type)
2. `router::close_position<CoinA, CoinB, FeeType>`

3.4 Non-Atomic Effective-Range Migration Preserves Stale Reward Weight

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`sources/pool.move, sources/rewarder.move`**DESCRIPTION:**

`update_effective_tick_range_internal` activates the new pool reward regime before all dependent positions are migrated. Only up to `limit` positions have local share metadata refreshed, so the pool can enter a mixed state where the pool uses the new range while some positions still keep stale local share.

```
fun update_effective_tick_range_internal<CoinA, CoinB, FeeType>(...):
  Option<ID> {
    ...
    pool.effective_tick_lower = eff_lower;
    pool.effective_tick_upper = eff_upper;
    pool.sqrt_price = sqrt_price;
    ...
    while (ct < limit && option::is_some<ID>(&cursor)) {
      ...
      let new_sh = calculate_position_share(...);
      update_position_share(..., new_sh);
      pi.effective_tick_lower = pool.effective_tick_lower;
      pi.effective_tick_upper = pool.effective_tick_upper;
      pi.sqrt_price = pool.sqrt_price;
      ...
    };
    ...
  }
```

then stores a mixed `pool.total_share`. Processed positions contribute recalculated share, while unprocessed positions still contribute stale share. Reward settlement uses this mixed denominator.

```
fun update_effective_tick_range_internal<CoinA, CoinB, FeeType>(...):
  Option<ID> {
    ...
```

```

    pool.total_share = new_ts;
    rewarder::set_pool_share(
        rewarder_manager,
        object::id<CLMMPool<CoinA, CoinB, FeeType>>(clmm_pool),
        pool.total_share,
    );
    ...
}

```

```

fun accumulate_pool_reward(
    rewarder: &mut Rewarder,
    pool_id: ID,
    pool_share: u128,
    now: u64,
): u128 {
    ...
    pool_info.acc_per_share =
        pool_info.acc_per_share + (reward_delta as u128) / pool_share;
    ...
}

```

`harvest` and `accumulated_position_rewards` directly trust `info.share` and continue accounting with the old reward weight.

```

public fun harvest<RewardCoin>(...) {
    ...
    let old_share = info.share;
    update_position_share(
        info,
        rewarder::pool_rewards_settle(rewarder_manager, rewarders,
        clmm_pool_id, clk),
        old_share,
    );
    ...
}

```

```

public fun accumulated_position_rewards(...) {
    ...
    let old_share = info.share;
    update_position_share(
        info,

```

```
        rewarder::pool_rewards_settle(rewarder_manager, rewarders,  
    clmm_pool_id, clk),  
        old_share,  
    );  
    ...  
}
```

IMPACT:

Positions that should lose reward weight under the new effective range can continue accruing and settling rewards with stale share while the pool has already switched to the new regime, causing reward leakage, mixed-denominator accounting, and unfair redistribution from correctly updated liquidity providers.

RECOMMENDATIONS:

Update the position share before any reward-affecting action when local range metadata differs from current pool state.

3.5 Emergency Withdrawal Is Disabled During Emergency State

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`sources/rewarder.move, sources/config.move`**DESCRIPTION:**

`emergent_withdraw` is intended as an admin only reward vault withdrawal path, but it still uses the normal `checked_package_version` gate. After `emergency_pause` sets `config.package_version` to a large sentinel value, `checked_package_version` starts rejecting the call, so the emergency withdrawal path becomes unavailable precisely during an emergency state.

```
public fun emergent_withdraw<RewardCoin>(...) {
    config::checked_package_version(global_config);
    let reward_type = type_name::with_defining_ids<RewardCoin>();
    assert!(bag::contains<TypeName>(&rewarder_manager.vault,
    reward_type), EInsufficientVaultBalance);
    ...
    balance::split<RewardCoin>(vault_balance, amount)
}
```

```
public fun checked_package_version(config: &GlobalConfig) {
    assert!(package_version() >= config.package_version,
    EPackageVersionDeprecate);
}
```

```
public fun emergency_pause(config: &mut GlobalConfig, ctx:
    &tx_context::TxContext) {
    check_emergency_pause_role(config, tx_context::sender(ctx));
    config.package_version = 9223372036854775808;
}
```

```
public fun emergency_unpause(config: &mut GlobalConfig, _: &AdminCap) {
```

```
    assert!(config.package_version == 9223372036854775808,
ENotEmergencyPaused);
    config.package_version = package_version();
}
```

IMPACT:

During an active emergency pause, admin cannot use the emergency reward vault withdrawal path without first unpausing the package. Temporarily restores normal checked flows before funds can be recovered, reducing the protocol's ability to contain an ongoing incident.

RECOMMENDATIONS:

Use a dedicated emergency paused check instead of `checked_package_version`.

```
public fun emergent_withdraw<RewardCoin>(
    _admin_cap: &config::AdminCap,
    global_config: &config::GlobalConfig,
    rewarder_manager: &mut RewarderManager,
    amount: u64,
): Balance<RewardCoin> {
-   config::checked_package_version(global_config);
    let reward_type = type_name::with_defining_ids<RewardCoin>();
    assert!(bag::contains<TypeName>(&rewarder_manager.vault,
reward_type), EInsufficientVaultBalance);
    let vault_balance = bag::borrow_mut<TypeName,
Balance<RewardCoin>>(
        &mut rewarder_manager.vault,
        reward_type,
    );
    assert!(balance::value<RewardCoin>(vault_balance) >= amount,
EInsufficientVaultBalance);
    balance::split<RewardCoin>(vault_balance, amount)
}
```

3.6 Unverified `sqrt_price` Can Distort Farming Share Accounting

SEVERITY:

MEDIUM

STATUS:

Fixed

PATH:`sources/pool.move`**DESCRIPTION:**

The vulnerable functions are `create_pool` and `update_effective_tick_range_internal`. Both functions accept `sqrt_price` as an external input and only validate that the tick derived from this value is inside the configured effective range:

```
check_effective_range(  
    effective_tick_lower,  
    effective_tick_upper,  
    sqrt_price,  
    clmm_pool::get_pool_tick_spacing<CoinA, CoinB, FeeType>(clmm_pool),  
);
```

Inside `check_effective_range`, the code calculates a tick from the provided `sqrt_price`:

```
let tick = math_tick::tick_index_from_sqrt_price(sqrt_price);  
assert!(i32::lt(eff_lower, tick) && i32::gt(eff_upper, tick),  
    EEffectiveRangeError);
```

This proves only that the supplied price falls within the selected range. It does not prove that `sqrt_price` equals the real current price of the referenced CLMM pool. The value is then stored in `Pool.sqrt_price` and later used by `calculate_position_share` to compute user reward weight. Since farming rewards depend on `share`, an incorrect but range-valid `sqrt_price` can cause positions to be overvalued or undervalued relative to the real CLMM state.

IMPACT:

A pool manager can intentionally or accidentally initialize or update a farming pool with a stale or incorrect `sqrt_price`. This can distort `calculate_position_share`, causing unfair reward

distribution across users. Positions near the chosen artificial price may receive excessive rewards, while positions that should be valuable at the real CLMM price may receive too little or zero share.

RECOMMENDATIONS:

Do not trust externally supplied `sqrt_price` for farming share accounting. In `create_pool` and `update_effective_tick_range_internal`, read the real price directly from the CLMM pool:

```
let sqrt_price = clmm_pool::get_pool_sqrt_price<CoinA, CoinB,  
    FeeType>(clmm_pool);
```

Then pass this value into `check_effective_range` and store it in `Pool.sqrt_price`. If manual price input must remain, add an assertion that the supplied value equals the live CLMM pool price, or is within a tightly bounded tolerance.

3.7 Emergency Withdraw Event Reports Pre-Withdrawal Vault Balance

SEVERITY: LOW**STATUS:** Fixed**PATH:**`sources/rewarder.move`**DESCRIPTION:**

`emergent_withdraw` emits `EmergentWithdrawEvent.after_amount` before the vault balance is reduced. the event records `balance::value(vault_balance)` prior to `balance::split`, so the emitted value is actually the pre-withdrawal amount.

```
public struct EmergentWithdrawEvent has copy, drop, store {
    reward_type: TypeName,
    withdraw_amount: u64,
    after_amount: u64,
}
```

```
public fun emergent_withdraw<RewardCoin>(...) {
    ...
    let vault_balance =
        bag::borrow_mut<TypeName, Balance<RewardCoin>>(
            &mut rewarder_manager.vault,
            reward_type,
        );
    assert!(balance::value<RewardCoin>(vault_balance) >= amount,
EInsufficientVaultBalance);
    let event = EmergentWithdrawEvent {
        reward_type: reward_type,
        withdraw_amount: amount,
        after_amount: balance::value<RewardCoin>(vault_balance),
    };
    emit<EmergentWithdrawEvent>(event);
    balance::split<RewardCoin>(vault_balance, amount)
}
```

`after_amount` correctly represents the post-operation vault balance in `DepositEvent`, but di-

verges from the established event semantics in [EmergentWithdrawEvent](#).

```
public struct DepositEvent has copy, drop {
    reward_type: TypeName,
    deposit_amount: u64,
    after_amount: u64,
}
```

```
public fun deposit_rewarder<RewardCoin>(...) {
    let event = DepositEvent {
        reward_type: reward_type,
        deposit_amount: deposit_amount,
        after_amount: balance::join<RewardCoin>(..., balance),
    };
    emit<DepositEvent>(event);
}
```

IMPACT:

Off-chain systems that rely on [EmergentWithdrawEvent](#).[after_amount](#) can overestimate the remaining reward vault balance after an emergency withdrawal.

RECOMMENDATIONS:

Emit the event after computing the post-withdrawal balance.

```
public fun emergent_withdraw<RewardCoin>(...) {
    ...
+   let after_amount = balance::value<RewardCoin>(vault_balance) -
    amount;
    let event = EmergentWithdrawEvent {
        reward_type: reward_type,
        withdraw_amount: amount,
-       after_amount: balance::value<RewardCoin>(vault_balance),
+       after_amount,
    };
    emit<EmergentWithdrawEvent>(event);
    balance::split<RewardCoin>(vault_balance, amount)
}
```

3.8 Update Pool Allocate Point Event Reports Post Update Value as Old

SEVERITY:

LOW

STATUS:

Fixed

PATH:

sources/pool.move, sources/rewarder.move

DESCRIPTION:

`update_pool_allocate_point` calls `rewarder::set_pool` before emitting `UpdatePoolAllocatePoint`. `set_pool` writes the new `allocate_point` state, so the subsequent `borrow_pool_allocate_point` read returns the updated value, not the pre update value.

```
public struct UpdatePoolAllocatePointEvent has copy, drop {
    pool_id: ID,
    clmm_pool_id: ID,
    old_allocate_point: u64,
    new_allocate_point: u64,
}
```

```
public fun update_pool_allocate_point<RewardCoin>(...) {
    config::check_pool_manager_role(global_config,
    tx_context::sender(ctx));
    rewarder::set_pool<RewardCoin>(
        rewarder_manager,
        pool.clmm_pool_id,
        allocate_point,
        clock,
    );
    emit(UpdatePoolAllocatePointEvent {
        pool_id: object::id<Pool>(pool),
        clmm_pool_id: pool.clmm_pool_id,
        old_allocate_point: rewarder::borrow_pool_allocate_point(
            rewarder_manager,
            type_name::with_defining_ids<RewardCoin>(),
            pool.clmm_pool_id,
        ),
        new_allocate_point: allocate_point,
    });
}
```

```
});  
}
```

```
public(package) fun set_pool<RewardCoin>(...) {  
    // ...  
    pool_rewarder_info.allocate_point = allocate_point;  
    pool_rewarder_info.acc_per_share  
}
```

IMPACT:

After every successful update, `old_allocate_point` and `new_allocate_point` in the event are therefore equal, and the true previous weight is never emitted.

RECOMMENDATIONS:

Read the previous allocate point before calling `set_pool`.

```
public fun update_pool_allocate_point<RewardCoin>(...) {  
    config::check_pool_manager_role(global_config,  
    tx_context::sender(ctx));  
+    let reward_type = type_name::with_defining_ids<RewardCoin>();  
+    let old_allocate_point = rewarder::borrow_pool_allocate_point(  
+        rewarder_manager,  
+        reward_type,  
+        pool.clmm_pool_id,  
+    );  
    rewarder::set_pool<RewardCoin>(  
        rewarder_manager,  
        pool.clmm_pool_id,  
        allocate_point,  
        clock,  
    );  
    emit(UpdatePoolAllocatePointEvent {  
        pool_id: object::id<Pool>(pool),  
        clmm_pool_id: pool.clmm_pool_id,  
-        old_allocate_point: rewarder::borrow_pool_allocate_point(  
-            rewarder_manager,  
-            type_name::with_defining_ids<RewardCoin>(),  
-            pool.clmm_pool_id,
```

```
-         ),  
+         old_allocate_point,  
         new_allocate_point: allocate_point,  
     });  
}
```

3.9 Missing CollectFeeEvent Reduces Fee Collection Observability

SEVERITY: LOW**STATUS:** Fixed**PATH:**`sources/pool.move`**DESCRIPTION:**

The `collect_fee` function collects accrued CLMM trading fees from the underlying `TurbosPositionNFT`, but it does not emit any protocol-level event after the collection succeeds. The function verifies the CLMM pool matches the wrapped NFT, calls `collect_with_return_`, and returns the collected balances directly:

```
let nft_id = object::id_address(&wrapped_position.nft);
let (ca, cb) = turbos_clmm::position_manager::collect_with_return_<CoinA,
  CoinB, FeeType>(
  clmm_pool,
  positions,
  &mut wrapped_position.nft,
  0xFFFFFFFFFFFFFFFF,
  0xFFFFFFFFFFFFFFFF,
  nft_id,
  clk.timestamp_ms() + 60000,
  clk,
  versioned,
  ctx,
);
(coin::into_balance<CoinA>(ca), coin::into_balance<CoinB>(cb))
```

Unlike deposit, withdraw, harvest, add liquidity, and remove liquidity flows, this fee-collection path has no corresponding event containing the wrapped position ID, underlying NFT ID, CLMM pool ID, and collected amounts. The absence of a `CollectFeeEvent` makes this user action less transparent than other state-changing operations in the farming wrapper.

IMPACT:

Off-chain indexers, analytics systems, monitoring bots, and user interfaces cannot reliably track fee

collections from the farming module's event stream. This weakens accounting reconciliation, user activity history, incident investigation, and protocol observability.

RECOMMENDATIONS:

Add a `CollectFeeEvent` and emit it after `collect_with_return_` succeeds.

3.10 Missing ClosePositionEvent Reduces Close Operation Observability

SEVERITY: LOW**STATUS:** Fixed**PATH:**`sources/pool.move`**DESCRIPTION:**

The vulnerable function is `close_position`. This function fully closes a wrapped farming position by first calling `withdraw`, then removing all remaining liquidity from the underlying CLMM position, and finally burning the underlying `TurbosPositionNFT`. Although `withdraw` emits a `WithdrawEvent`, that event only records the farming-layer withdrawal and does not describe the full close operation.

```
let mut nft = withdraw(global_config, rewarder_manager, pool,
    wrapped_position, clk);
...
let (ca, cb) =
    turbos_clmm::position_manager::decrease_liquidity_with_return(...);
...
turbos_clmm::position_manager::burn<CoinA, CoinB, FeeType>(
    positions,
    nft,
    versioned,
    ctx,
);
```

The current `close_position` function does not emit its own event showing how much liquidity was closed, how many `CoinA` and `CoinB` were returned, or which underlying NFT was burned. As a result, off-chain systems that rely on farming-module events cannot distinguish a normal farming withdrawal from a full position close and burn.

IMPACT:

This issue does not create an authorization bypass or direct fund loss, because the function still consumes the user-owned `WrappedPositionNFT` and validates the CLMM pool relationship.

RECOMMENDATIONS:

Add a dedicated `ClosePositionEvent` emitted after successful liquidity removal and before or after the underlying NFT burn.

3.11 Missing Post-Condition Check After Adding Liquidity

SEVERITY: LOW**STATUS:** Fixed**PATH:**`sources/pool.move`**DESCRIPTION:**

The vulnerable function is `add_liquidity`. The function validates that the requested `delta_liquidity` is greater than zero, calculates the required token amounts, and then calls the underlying CLMM `increase_liquidity_with_return` function.

```
assert!(delta_liquidity > 0, EPositionLiquidityInvalid);
...
let (coin_a_left, coin_b_left) =
  turbos_clmm::position_manager::increase_liquidity_with_return<CoinA,
  CoinB, FeeType>(
    clmm_pool,
    positions,
    vector::singleton(coin_a),
    vector::singleton(coin_b),
    &mut wrapped_position.nft,
    need_a,
    need_b,
    need_a,
    need_b,
    deadline,
    clk,
    versioned,
    ctx,
  );
```

After the call, the function queries the updated position liquidity and updates farming accounting.

```
let (_, _, new_liquidity) =
  turbos_clmm::position_manager::get_position_info(positions,
  nft_address);
```

```
let new_share = update_position_liquidity(  
    rewarder_manager,  
    pool,  
    wrapped_position,  
    new_liquidity,  
    clk,  
);
```

However, it does not explicitly verify that `new_liquidity > old_liquidity`, or that the liquidity increased by the requested `delta_liquidity`. If the downstream CLMM behavior changes, returns an unexpected state, or a future integration path produces a no-op/partial liquidity update, the farming module may still update accounting and emit an `AddLiquidityEvent` as if liquidity was successfully added.

IMPACT:

The farming wrapper relies on an external module side effect without validating the post-condition it depends on.

RECOMMENDATIONS:

After reading `new_liquidity`, add an explicit post-condition check before updating farming accounting. At minimum, require.

```
assert!(new_liquidity > old_liquidity, EPositionLiquidityInvalid);
```

3.12 Missing Non-Zero Amount Validation in add_liquidity_fix_coin

SEVERITY:

LOW

STATUS:

Fixed

PATH:

sources/pool.move

DESCRIPTION:

The vulnerable function is `add_liquidity_fix_coin`. This function is intended to add liquidity to an existing wrapped CLMM position using caller-supplied `amount_a` and `amount_b`, with one side selected as the fixed input by `fix_amount_a`. However, the function does not validate that at least one input amount is non-zero before calling the underlying CLMM liquidity function.

```
let (min_a, min_b) = if (fix_amount_a) { (amount_a, 0) } else { (0,
    amount_b) };

let (coin_a_left, coin_b_left) =
    turbos_clmm::position_manager::increase_liquidity_with_return<CoinA,
    CoinB, FeeType>(
        clmm_pool,
        positions,
        vector::singleton(coin_a),
        vector::singleton(coin_b),
        &mut wrapped_position.nft,
        amount_a,
        amount_b,
        min_a,
        min_b,
        deadline,
        clk,
        versioned,
        ctx,
    );
```

If both `amount_a == 0` and `amount_b == 0`, the wrapper still forwards the call to the CLMM module. Depending on downstream behavior, this may abort, become a no-op, or return unchanged liquidity. The farming wrapper then reads `new_liquidity`, updates accounting, and emits

`AddLiquidityFixCoinEvent` without first enforcing that the input represents a meaningful liquidity addition.

IMPACT:

Allowing zero-amount calls weakens input validation and can create confusing or noisy state-transition attempts.

RECOMMENDATIONS:

Add an explicit non-zero input validation before calling the CLMM position manager:

```
assert!(amount_a > 0 || amount_b > 0, EPositionLiquidityInvalid);
```

For stronger correctness, also validate the post-condition after reading `new_liquidity`:

```
assert!(new_liquidity > old_liquidity, EPositionLiquidityInvalid);
```

4. CONCLUSION

In this audit, we thoroughly analyzed **Turbos Farming** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec